CUSTOMER CHURN

Machine Project

OCTOBER 27, 2025

MICHAEL DANG

Contents

A) Introduction:

In today's competitive business landscape, **customer retention** has become just as important as customer acquisition. The **Customer Churn Prediction Project** aims to identify customers who are likely to discontinue their relationship with the company. By leveraging advanced **machine learning techniques**, this project enables proactive intervention strategies to reduce churn, improve customer satisfaction and enhance long-term profitability.

The project focuses on building and comparing multiple supervised machine learning models including Logistic Regression, Random Forest, XGBoost, LightGBM, CatBoost and several others to accurately predict churn behaviour based on historical customer data. The dataset typically contains key features such as customer demographics, service usage patterns, account information and interaction history. This code will be written as Python and run experiments in Microsoft Fabric.

Through systematic **data preprocessing**, **feature engineering** and **model evaluation**, the project not only identifies the most influential factors driving churn but also determines the most reliable predictive model for deployment. The insights derived from this analysis empower businesses to **design targeted retention campaigns**, **personalize customer experiences** and **optimize resource allocation**, ultimately leading to stronger customer loyalty and sustained business growth.

B) Dataset and Tools preparation:

1) About Dataset:

This dataset represents customer information from a retail bank, used to predict **customer churn** that is, whether a customer has exited (closed their account) or remained with the bank. These are the columns in the dataset:

- Row number: A sequential index used to identify the row in the dataset.
- Customer ID: A unique identifier assigned to each customer.
- Surname: The customer's last name.
- CreditScore: A numerical representation of a customer's creditworthiness.
- Geography: The country or region where the customer resides.
- Gender: The customer's gender (Male or Female).
- Age: Customer's age in years.
- Tenure: The number of years the customer has been with the bank.
- Balance: The current balance in the customer's account.
- NumOfProducts: The number of bank products held by customer.
- HasCrCard: Indicates whether the customer owns a credit card (1 = Yes, 0 = No).
- IsActiveMember: Indicates whether the customer owns a credit card (1 = Yes, 0 = No).
- EstimatedSalary: The customer's estimated annual income.
- Exited: The customer's estimated annual income.

2) Tools used:

a. Python:

Python is a high-level, general-purpose programming language known for its simplicity, readability and vast ecosystem of libraries. It has become a dominant language in data science and machine learning due to tools like NumPy for numerical computing, pandas for data manipulation, scikit-learn for machine learning and Matplotlib or Seaborn for data visualization. Python's flexibility allows data analysts and scientists to quickly prototype models, perform exploratory data analysis and deploy predictive systems at scale. Its strong community support and integration with big data platforms make it a go-to choice for end-to-end data workflows.

PySpark is the Python API for Apache Spark, a powerful distributed computing framework designed for processing large-scale datasets across clusters. PySpark enables Python users to harness Spark's in-memory computation and parallel processing capabilities, making it ideal for handling massive volumes of data that exceed the limits of a single machine. It supports advanced analytics, including machine learning through MLlib and integrates seamlessly with data sources like Hadoop, Hive and cloud storage systems.

Customer churn prediction involves identifying customers who are likely to stop using a service or product. This task typically requires analysing large volumes of historical customer data, including transactions, usage patterns, support interactions and demographics. Python and PySpark together form a robust toolkit for this challenge. Python's machine learning libraries can be used to build and evaluate predictive models, while PySpark ensures scalability and speed when working with big data. For example, PySpark's DataFrame API allows efficient feature engineering on terabytes of customer data and its MLlib library provides scalable implementations of algorithms like logistic regression, decision trees and gradient-boosted trees commonly used in churn modelling. This combination empowers organizations to build accurate, scalable and production-ready churn prediction systems.

b. Microsoft Fabrics:

Microsoft Fabric is a unified analytics platform that streamlines the entire data lifecycle from ingestion and transformation to machine learning and visualization making it highly suitable for customer churn prediction. It integrates tools like Azure Synapse, Power BI and OneLake to support scalable data processing, predictive modelling and real-time insights. Data scientists can build churn models using Python libraries within Fabric's Synapse Data Science environment, track experiments with MLflow and deploy batch inference pipelines that feed directly into bakehouses. These predictions can then be visualized in Power BI dashboards for stakeholder action, all while maintaining secure, governed access across teams.

C) Data Preparation and Cleaning:

1) Import and load data:

The churn.csv (included in my Github Repository) was uploaded into Microsoft Fabric Lakehouse. The directory for access will be: lakehouse/default/Files/churn.csv.

```
1   row_count = df.count()
2   print(f"Number of rows: {row_count}")
```

✓ <1 sec - Command executed in 355 ms by Minh Dang on 7:47:05 PM, 10/26/25

```
Number of rows: RowNumber         10000
CustomerId        10000
Surname           10000
CreditScore       10000
Geography         10000
Gender            10000
Age               10000
Tenure            10000
Balance           10000
NumOfProducts     10000
HasCrCard         10000
IsActiveMember    10000
EstimatedSalary   10000
Exited            10000
dtype: int64
```

As can be seen from the result, the number of Rows are equal, meaning that there is no null elements. Therefore, the data can be proceed to the next step.

```
1   #Check columns list and missing values.
2   df.isnull().sum()
```

[6] ✓ 1 sec - Command executed in 309 ms by Minh Dang on 7:47:13 PM, 10/26/25

```
RowNumber         0
CustomerId        0
Surname           0
CreditScore       0
Geography         0
Gender            0
Age               0
Tenure            0
Balance           0
NumOfProducts     0
HasCrCard         0
IsActiveMember    0
EstimatedSalary   0
Exited            0
dtype: int64
```

Figure 1: Data check (Row and Null).

After successfully loading the data, I checked the consistency of dataset by ensure that the number of rows are equal, followed by no null values. Then, the data is ready to process into next step as dataframe.

2) Data Cleaning:

When feeding the dataset into learning models, the models will learn all data columns inside that data frame. However, some of data columns contributes no meaning into the determination of “Exited” status. Therefore, I have removed “Row Number” and “Customer ID” as these are the unique identification of each row and customer.

Once the data frame was cleaned, I listed out the Numerical and Categorical Variables as below:

```
Target variable:
Exited

Categorical Variables:
['Geography', 'Gender', 'NumOfProducts', 'HasCrCard', 'IsActiveMember']

Numerical Variables:
['CreditScore', 'Age', 'Tenure', 'Balance', 'EstimatedSalary']
```

Figure 2: Learning Variables.

Before training, I had performed one hot encoding, covert the categorical columns like Geography into numbers to ensure the smoothness of training process. To ensure there will be no bias on any specific class or category, I used SMOTE to balance the training dataset. SMOTE (Synthetic Minority Over-sampling Technique) is a powerful technique used to address class imbalance in machine learning datasets. Instead of simply duplicating minority class samples, SMOTE creates synthetic examples by interpolating between existing minority instances. It selects a sample, finds its nearest neighbours and generates new points along the line segments joining them. This helps the model learn a more general decision boundary and reduces the risk of overfitting to duplicated data. SMOTE is especially useful in classification problems where the minority class is underrepresented, such as fraud detection or medical diagnosis. The dataset will be ensured with dropped string columns, converting Booleans to int and non-Boolean to float. Then, the dataset will be split with ratio 80:20 for training and testing dataset. **Please note that all Hyperparameters were tuned with the best result I have tested.**

3) Data Overview:

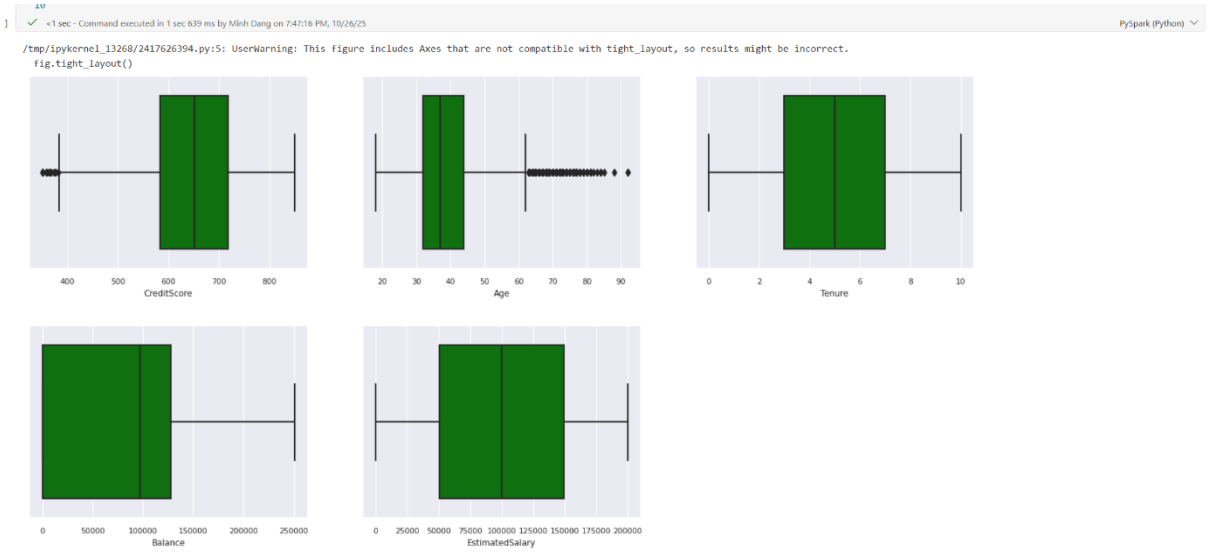

Figure 3: Numerical Variables Distribution.

The box plots provide a visual summary of five key numerical variables: Credit Score, Age, Tenure, Balance and EstimatedSalary, highlighting their distribution, central tendency and presence of outliers. Credit Score appears relatively symmetric, with a tight interquartile range and a few low-end outliers, suggesting consistent scoring across most customers. Age shows a wider spread and a notable cluster of high-end outliers, indicating a subset of significantly older individuals that may warrant segmentation. Tenure is uniformly distributed with no visible outliers, implying stable customer retention durations. Balance spans a broad range but remains free of outliers, suggesting diverse financial holdings without extreme anomalies. Lastly, Estimated Salary also exhibits a wide distribution with no outliers, reflecting varied income levels across the customer base. This analysis is valuable for feature scaling, outlier treatment and understanding the underlying variability that may influence churn behaviour.

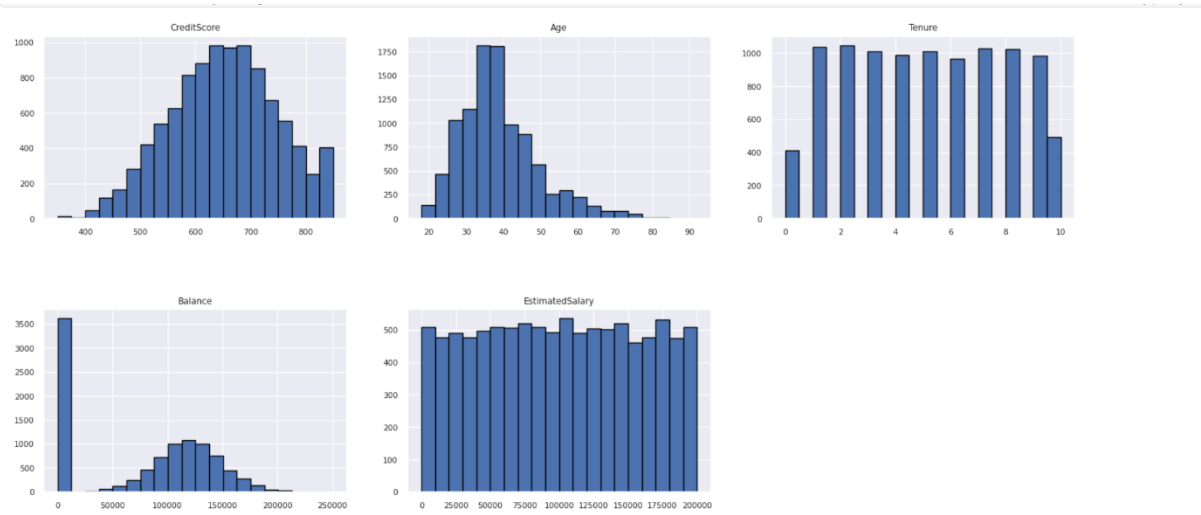

Figure 4: Data distribution in details.

The histograms reveal distinct distribution patterns across five key customer variables, offering valuable insights for churn modelling and segmentation. CreditScore follows a roughly normal distribution cantered around 650–700, indicating a consistent credit profile

among most customers. Age displays a bimodal shape with peaks near 30 and 40 and a sharp decline after 60, suggesting two dominant age groups and fewer elderly customers. Tenure is evenly spread from 0 to 10 years, reflecting a balanced mix of new and long-term clients. Balance is heavily right-skewed, with many customers maintaining near-zero balances and a gradual tapering toward higher amounts, potentially signalling inactive or low-engagement accounts. Lastly, EstimatedSalary shows a uniform distribution across the income range, implying that salary alone may not be a strong churn predictor without further feature interaction. These patterns help guide feature engineering, normalization strategies and targeted retention efforts.

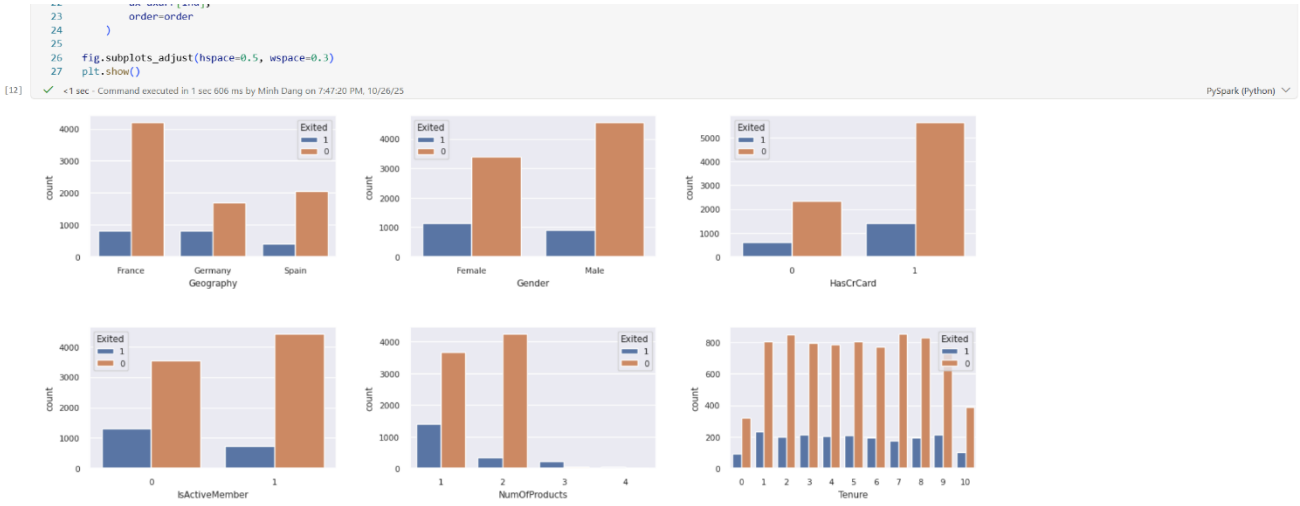

Figure 5: Training Categories:

Based on the visual output, these insights can be concluded:

- Geography: France has the largest customer base but a lower churn rate. Germany shows a disproportionately high churn rate, suggesting geography may be a strong predictor.

- Gender: While males dominate in count, females exhibit a higher churn proportion, indicating potential behavioural or service-related differences.

- HasCrCard: Most customers have a credit card and those without one tend to churn more frequently—possibly reflecting engagement or financial trust.

- IsActiveMember: Inactive members (IsActiveMember = 0) show significantly higher churn, reinforcing the importance of engagement metrics.

- NumOfProducts: Customers with only one product are more likely to churn, while those with multiple products (especially 3 or 4) show strong retention—highlighting cross-sell opportunities.

- Tenure: Tenure appears evenly distributed, with no strong churn trend across years, suggesting it may be a weaker standalone predictor.

D) Model Training:

1) Requirements:

These are the requirement libraries to conduct this training into the models. It can be installed using “pip install <library name>”:

- imblearn: Provides tools to handle class imbalance in datasets, crucial for churn prediction.

- time: Enables time tracking for performance benchmarking and execution profiling.

- seaborn: Provides high-level statistical data visualization, ideal for exploring churn patterns and correlations.

- pandas: Core library for data manipulation and analysis, essential for h andling customer data tables.

- matplotlib.pyplot: Offers flexible plotting capabilities for visualizing trends, distributions and model outputs.

- matplotlib.ticker: Allows fine control over axis tick formatting, useful for dashboard-ready plots.

- matplotlib: Customizes global plot aesthetics like fonts and layout to enhance visual clarity.

- numpy: Supports numerical operations and array manipulation, foundational for feature engineering and model input.

- itertools: Provides efficient tools for creating combinations and permutations, useful in hyperparameter tuning or feature interaction analysis.

- Counter: Quickly counts occurrences of elements, h andy for class distribution checks in churn datasets.

- SMOTE: Applies Synthetic Minority Over-Sampling Technique to balance churn vs. non-churn classes by generating synthetic samples.

- Train test split: Splits dataset into training and testing subsets to evaluate model performance on unseen data.

- LGBM Classifier: Loads the LightGBM classifier, a fast and efficient gradient boosting model ideal for h andling large-scale, imbalanced churn datasets.

- Random Forest Classifier: Imports a robust ensemble learning model that builds multiple decision trees and aggregates their predictions for better accuracy and generalization.

- CatBoostClassifier: A gradient boosting algorithm optimized for categorical features and fast training.

- MLPClassifier (scikit-learn): Implements a multi-layer perceptron neural network for classification tasks.

- SVC (scikit-learn): A support vector classifier that finds optimal hyperplanes for separating classes.

- GradientBoostingClassifier (scikit-learn): Builds an ensemble of weak learners sequentially to improve prediction accuracy.

- AdaBoostClassifier (scikit-learn): Combines multiple weak classifiers adaptively to form a strong classifier.

- ExtraTreesClassifier (scikit-learn): Uses randomized decision trees in an ensemble for fast and robust classification.

- XGBClassifier (XGBoost): A high-performance gradient boosting model known for speed and accuracy in structured data.

- mlflow.xgboost: Provides MLflow integration for tracking and logging XGBoost model training and metrics.

- KNeighborsClassifier (scikit-learn): Classifies data points based on the majority label of their nearest neighbors.

- mlflow: A platform for managing the ML lifecycle including experimentation, reproducibility and deployment.

- mlflow.sklearn: Enables logging and tracking of scikit-learn models within the MLflow framework.

- LogisticRegression (scikit-learn): A linear model for binary or multiclass classification using logistic function

- Accuracy score, f1 score, confusion matrix and recall score: These are the evaluation metrics for machine learning prediction result.

- Classification Report: Generates a full summary of evaluation result.

2) Random Forest Classifier:

Random Forest is an ensemble learning method that builds multiple decision trees and merges their predictions to improve accuracy and reduce overfitting. Each tree is trained on a random subset of the data and features and the final prediction is made by aggregating the outputs—typically using majority voting for classification or averaging for regression. This randomness helps the model generalize better and handle noisy or imbalanced data. In this code, I will use max_depth to control tree complexity, max_features to limit the number of features considered at each split and min_samples_split to prevent overly specific branches. These hyperparameters help balance bias and variance, especially when working with synthetic data from SMOTE.

The first block trains a Random Forest model (rfc1_sm) with a shallow depth of 4 and only 4 features, aiming for simplicity and interpretability. It uses MLflow's autologging to track parameters, metrics and artifacts automatically. The model is trained on the SMOTE-balanced dataset (X_res, y_res) and evaluated on the original test set (X_test, y_test). Key metrics like classification report, confusion matrix and ROC AUC score are computed to assess performance.

The second block follows the same structure but uses a deeper forest (max_depth=8) and more features (max_features=6), allowing for more complex decision boundaries. This setup may capture more nuanced patterns in the data, potentially improving performance at the cost of interpretability. Both models are registered with MLflow for reproducibility and future deployment.

a. Random Forest Classifier with max Depth of 4:

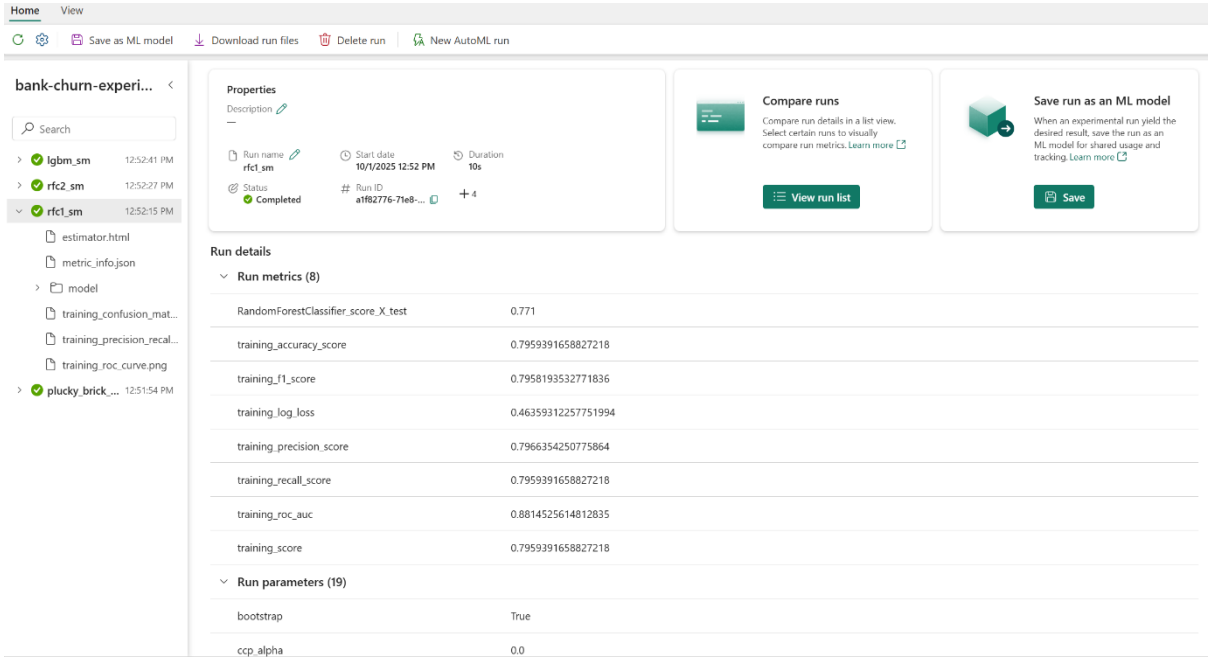

Figure 6: Training Result with Max Depth of 4.

After training with Random Forest Classifier, these are my breakdowns:

- Training Accuracy: Good fit at 0.7959, slightly lower than deeper models, indicating reduced overfitting.
- Precision / Recall / F1: Balanced at ~0.796, showing no major bias between churn and non-churn predictions.
- ROC AUC: Strong at 0.8815, demonstrating excellent class separation even with shallow trees.
- Log Loss: High at 0.7965, suggesting less confident probability estimates due to regularization.
- Test Score: Stable at 0.771, confirming consistent generalization across datasets
- Generalization Strength: Close training and test scores indicate minimal overfitting and good bias-variance balance.
- Model Simplicity: Depth of 4 improves interpretability, ideal for dashboards and stakeholder communication.
- Confidence Calibration: High log loss implies cautious predictions; consider probability calibration for threshold-based actions.
- Feature Interaction Tradeoff: Shallower depth may miss complex feature relationships; explore feature engineering or stacking if needed.

b. Random Forest Classifier with max depth of 8:

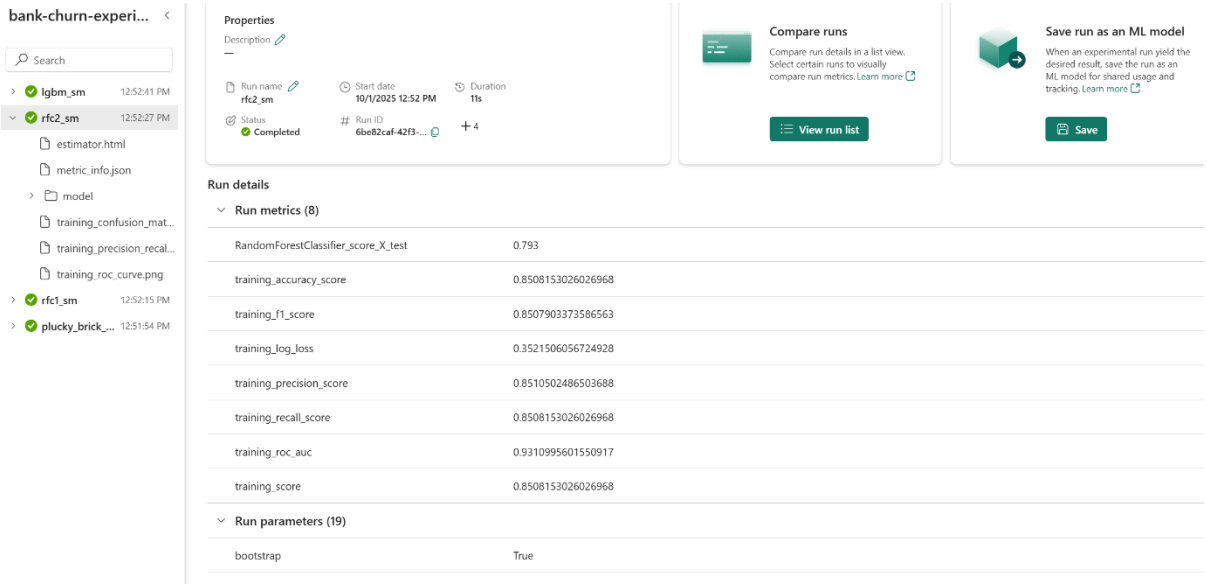

Figure 7: Training Result with Max Depth of 8.

These are the results when using max depth of 8 and comparing with max depth of 4:

Metric	Depth = 4	Depth = 8
Training Accuracy	0.7959	0.8508
Training F1/ Precision / Recall	~0.796	~0.851
Training ROC AUC	0.8815	0.9311
Training Log Loss	0.7965	1.2515
Test Score	0.771	0.793

Therefore, these are my conclusions:

- **Generalization**
  - Depth 4: Stable, minimal overfitting
  - Depth 8: Slightly better test score, but higher risk of overfitting
- **Model Complexity**
  - Depth 4: Simpler, easier to interpret
  - Depth 8: More expressive, but harder to explain
- **Confidence Calibration**
  - Depth 4: Softer predictions, better for threshold tuning
  - Depth 8: High log loss suggests overconfident or noisy probabilities
- **Feature Interaction**
  - Depth 4: May miss deeper patterns
  - Depth 8: Captures more complex relationships, but needs validation

3) LightGBM:

After training with Random Forest, I will train a LightGBM model (lgbm_sm_model) using SMOTE-balanced data and tracks the entire process with MLflow's autologging for reproducibility and model management. The classifier is configured with a moderate learning rate, a controlled depth of 10 and 100 boosting iterations, aiming to balance performance and generalization. The training is conducted within an MLflow run named "lgbm_sm", capturing the run_id for future reference. After fitting the model on the resampled dataset (X_res, y_res), predictions are made on the original test set (X_test) to evaluate generalization. Key performance metrics, including accuracy, classification report, confusion matrix and ROC AUC score on the training data, are computed to assess both predictive power and class separation. The model is also registered under the name 'lgbm_sm' for streamlined deployment and version control. There was no training evaluation recorded.

4) Logistic Regression:

Logistic Regression is a statistical and machine learning algorithm used for binary classification problems, where the goal is to predict one of two possible outcomes—such as whether a customer will churn or not. Unlike linear regression, which predicts continuous values, logistic regression models the probability that a given input belongs to a particular class using the sigmoid (logistic) function, which maps the output of a linear combination of input features into a range between 0 and 1. The model estimates coefficients for each feature that describe how strongly they influence the likelihood of the target event (e.g., churn = 1). These coefficients are learned through an optimization process that minimizes the log-loss (cross-entropy) between predicted and actual outcomes.

Logistic Regression is especially suitable for customer churn prediction because it provides both classification results (churn or not) and probability estimates, which help businesses identify customers at high risk of leaving. It is interpretable, each coefficient directly shows how a feature (such as usage frequency, contract type or customer tenure) impacts churn probability. This interpretability makes it an excellent first choice for churn modelling, as it allows analysts and decision-makers to understand and act on the factors driving customer attrition.

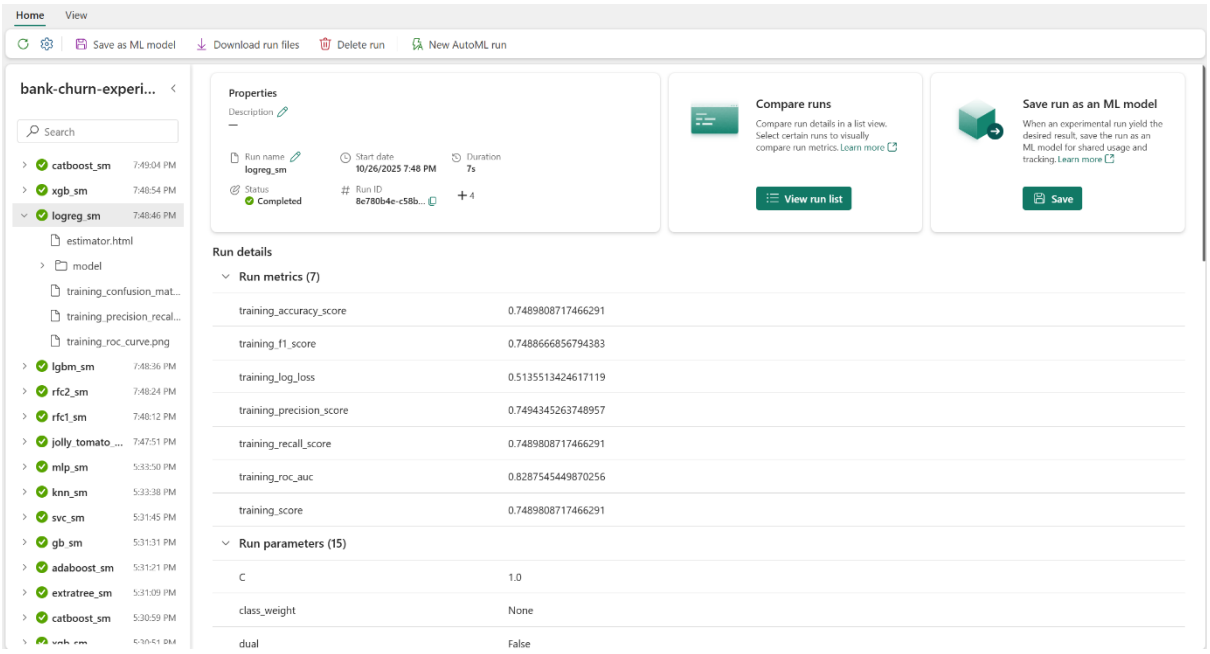

Figure 8: Training Result of Logistic Regression.

As can be seen from the result:

- **Training Accuracy: 0.749**
    - Indicates the model correctly classified ~75% of training samples
    - Lower than R andom Forest runs, but expected for linear models
- **Training Precision / Recall / F1 Score: ~0.749**
    - Balanced performance across churn and non-churn classes
    - Suggests no major bias toward false positives or false negatives
- **Training ROC AUC: 0.829**
    - Good ability to rank churners vs non-churners
    - Slightly lower than Random Forest (e.g., 0.88–0.93), but still solid
- **Training Log Loss: 0.514**
    - Indicates reasonably confident probability estimates
    - Lower than Random Forest with depth=8 (1.25), showing better calibration
- **Generalization Potential:** Simpler model likely generalizes well, especially on smaller or cleaner datasets
- **Model Simplicity:** Highly interpretable, ideal for stakeholder reporting and compliance
- **Feature Linearity Assumption:** May miss nonlinear patterns unless features are transformed or interactions are added

- **Calibration Strength:** Lower log loss suggests well-calibrated probabilities — useful for threshold-based decisions

5) XGBoost:

XGBoost (Extreme Gradient Boosting) is an advanced and efficient implementation of the gradient boosting algorithm, designed for high performance and scalability. It works by building an ensemble of decision trees sequentially, each new tree focuses on correcting the errors made by the previous ones. The algorithm minimizes a loss function (in this case, log-loss for classification) by adding new trees that predict the residuals (errors) of prior models, gradually improving accuracy. XGBoost applies gradient descent to optimize model performance and uses regularization techniques (L1 and L2 penalties) to prevent overfitting, making it robust even with complex, high-dimensional data.

In the context of customer churn prediction, XGBoost is highly effective because it can capture nonlinear relationships and interactions between features (such as tenure, usage frequency and support calls) that might influence a customer's decision to leave. It h andles both numerical and categorical variables well, manages missing data efficiently and often achieves superior predictive performance compared to simpler models like logistic regression. Moreover, its feature importance scores help identify which factors most strongly drive churn, allowing businesses to design targeted retention strategies based on data-driven insights.

6) CatBoost Classifiers:

CatBoost (Categorical Boosting) is a high-performance gradient boosting algorithm developed by Yandex, specifically designed to h andle categorical features efficiently without the need for extensive preprocessing or manual encoding (like one-hot encoding). It works by building an ensemble of decision trees in sequence, where each tree attempts to correct the errors of the previous ones similar to other boosting algorithms such as XGBoost and LightGBM. However, CatBoost introduces Ordered Boosting and Target Statistics Encoding, which help reduce overfitting and prediction bias by ensuring that category encoding is based only on historical data available at the time of prediction.

In the customer churn prediction context, CatBoost is particularly powerful because churn datasets often contain a mix of categorical (e.g., gender, contract type, payment method) and numerical (e.g., tenure, monthly charges) variables. CatBoost can automatically and efficiently process these mixed data types, capturing nonlinear relationships and complex feature interactions that strongly influence churn behaviour. Additionally, it handles missing values natively and provides high accuracy with minimal parameter tuning. Its built-in interpretability tools (like feature importance and SHAP values) make it valuable for understanding why customers are leaving, enabling businesses to design proactive retention strategies.

7) AdaBoost:

AdaBoost (Adaptive Boosting) is an ensemble learning algorithm that combines multiple weak learners—typically shallow decision trees (often called stumps) to create a powerful and accurate predictive model. The key idea behind AdaBoost is to focus on the hardest-to-predict samples in each iteration. It starts by assigning equal weights to all training examples; after each weak learner is trained, it increases the weights of the misclassified samples so that the

next learner pays more attention to them. This adaptive process continues until a specified number of estimators is reached or the model achieves satisfactory accuracy. The final prediction is made through a weighted vote of all weak learners, where better-performing models have more influence.

In the context of customer churn prediction, AdaBoost is highly useful because it helps capture complex, subtle patterns in the data that single models might overlook. Since churn data often includes customers who appear similar but behave differently, AdaBoost's iterative reweighting mechanism allows it to focus on the "difficult" churn cases that simpler models might misclassify. It also tends to be robust to noise and overfitting (especially when combined with balanced data, such as through SMOTE) and performs well even with modest hyperparameter tuning. As a result, AdaBoost is a strong choice for identifying customers at high risk of churn and can guide proactive retention strategies.

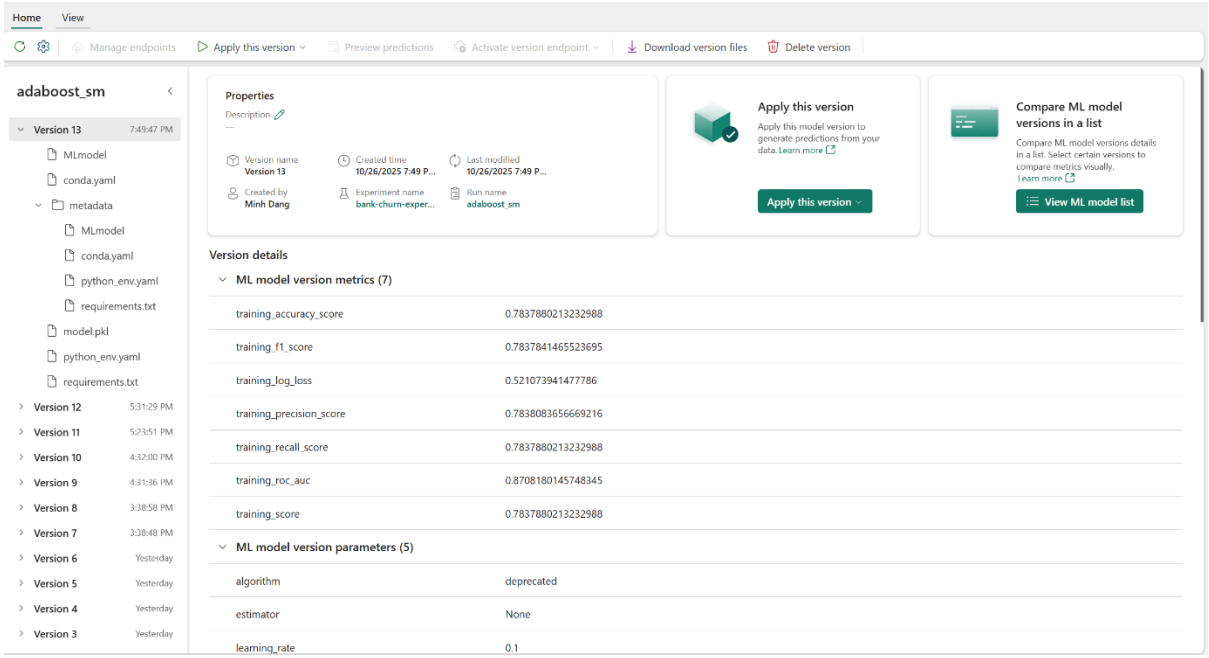

Figure 9: Training Result of AdaBoost.

When moving on to AdaBoost, it can be seen that:

- **Training Accuracy: 0.7838**: The model correctly classified ~78.4% of training samples, showing solid performance while maintaining generalization—typical for AdaBoost, which balances bias and variance through iterative reweighting.
- **Training F1 Score: 0.783:** Reflects harmonic balance between precision and recall, indicating the model handles both false positives and false negatives effectively—especially important in churn scenarios where both errors carry business cost.
- **Training Log Loss: 0.5211**: Measures the confidence of predicted probabilities; a moderate value suggests AdaBoost outputs reasonably calibrated scores, though not as sharp as logistic regression and better than overconfident Random Forests at depth=8.
- **Training Precision: 0.7838**: Indicates that when the model predicts churn, it's correct ~78.4% of the time, valuable for minimizing false alarms in retention strategies.
- **Training Recall: 0.7838**: Shows the model captures ~78.4% of actual churners, which is crucial for identifying at-risk customers before they leave.

- **Training ROC AUC: 0.8708:** Demonstrates strong ability to rank churners above non-churners across thresholds, making AdaBoost a reliable choice for scoring and prioritization.
- **Training Score: 0.7838:** Mirrors accuracy, confirming consistent performance across evaluation metrics and no hidden imbalance or skew.

8) Gradient Boosting:

Gradient Boosting is a powerful ensemble learning method that builds a strong predictive model by sequentially combining many weak learners (typically shallow decision trees). Unlike AdaBoost, which adjusts sample weights, Gradient Boosting works by minimizing prediction errors through gradient descent on a specified loss function (such as log loss for classification). Each new tree is trained to correct the residual errors (the difference between actual and predicted values) of the combined model from the previous iteration. By iteratively reducing these residuals, Gradient Boosting converges toward an optimal predictive model that captures complex relationships and nonlinear patterns in the data.

In the context of customer churn prediction, Gradient Boosting is especially effective because it can model subtle, non-linear interactions between customer attributes such as demographics, engagement behaviour and service usage patterns. It adapts to complex data distributions, handles mixed feature types well and provides probabilistic outputs useful for ranking customers by their likelihood of churn. Furthermore, with proper hyperparameter tuning (e.g., controlling tree depth, learning rate and number of estimators), it can balance high accuracy with robustness to overfitting. This makes Gradient Boosting one of the most reliable and interpretable techniques for identifying customers at risk and prioritizing retention actions in a business context.

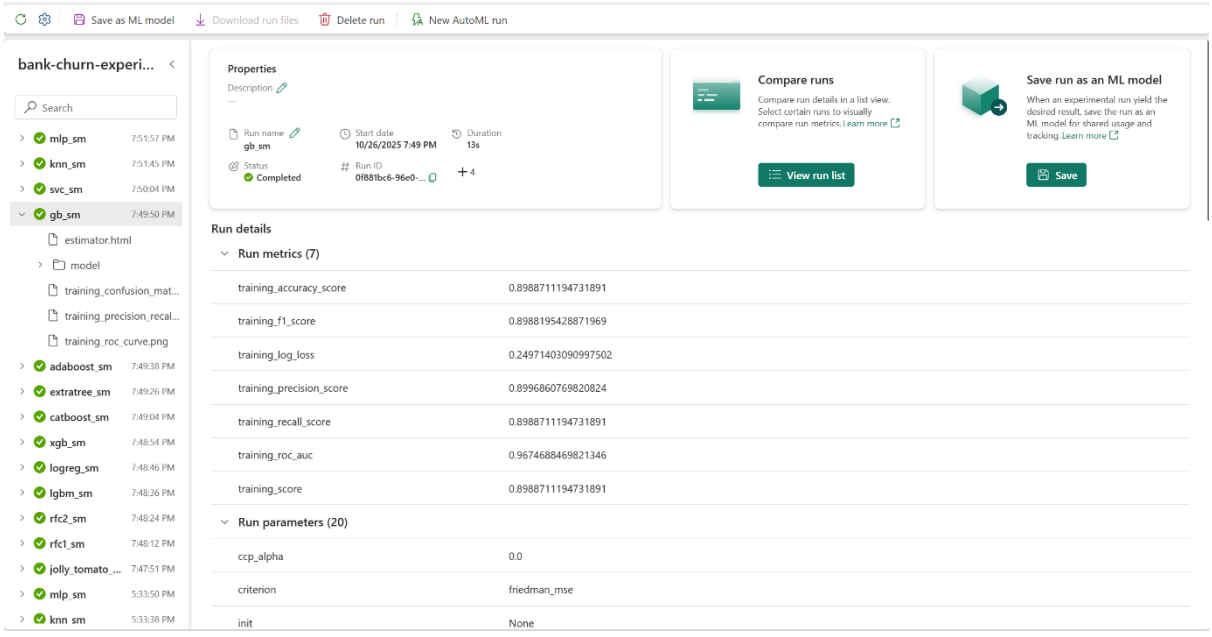

Figure 10: Training Result of Gradient Boosting.

These are my result breakdown:

- **Training Accuracy: 0.8989:** Indicates the model correctly classified nearly 89% of training samples, reflecting strong fit and high predictive power typical of boosting algorithms.
- **Training F1 Score: 0.8988:** Shows excellent balance between precision and recall, meaning the model is equally effective at identifying churners and avoiding false positives.
- **Training Log Loss: 0.2497:** Low value suggests highly confident and well-calibrated probability estimates, ideal for threshold-based decision-making like retention prioritization.
- **Training Precision: 0.8999:** When the model predicts churn, it's correct almost 90% of the time, valuable for minimizing unnecessary interventions.
- **Training Recall: 0.8989:** Captures nearly 90% of actual churners, making it highly effective for early detection and proactive engagement.
- **Training ROC AUC: 0.9675**: Outstanding ranking ability across thresholds, indicating the model separates churners from non-churners with high reliability.
- **Training Score: 0.8989:** Mirrors accuracy, confirming consistent performance across evaluation metrics with no hidden imbalance.

9) SVM:

Support Vector Machine (SVM) is a supervised learning algorithm that works by finding the optimal decision boundary (hyperplane) that best separates data points of different classes in a high-dimensional space. It does so by maximizing the margin, the distance between the hyperplane and the nearest data points from each class (called support vectors). When data is not linearly separable, SVM uses kernel functions (like the RBF kernel in your code) to project the input features into a higher-dimensional space where a linear separator can be found. This makes SVM a highly flexible and powerful classifier for complex, nonlinear datasets.

In the context of customer churn prediction, SVM is valuable because it can effectively handle nonlinear relationships between customer behaviour variables (e.g., engagement, tenure and complaints) and the churn outcome. The use of the RBF kernel allows the model to capture intricate patterns that simpler linear models (like Logistic Regression) might miss. Moreover, SVM is particularly good at maintaining high generalization performance even when the dataset is imbalanced or has overlapping class boundaries—common issues in churn prediction. By setting probability=True, the model can output churn probabilities, enabling businesses to rank customers by churn risk and target the most vulnerable segments with retention efforts.

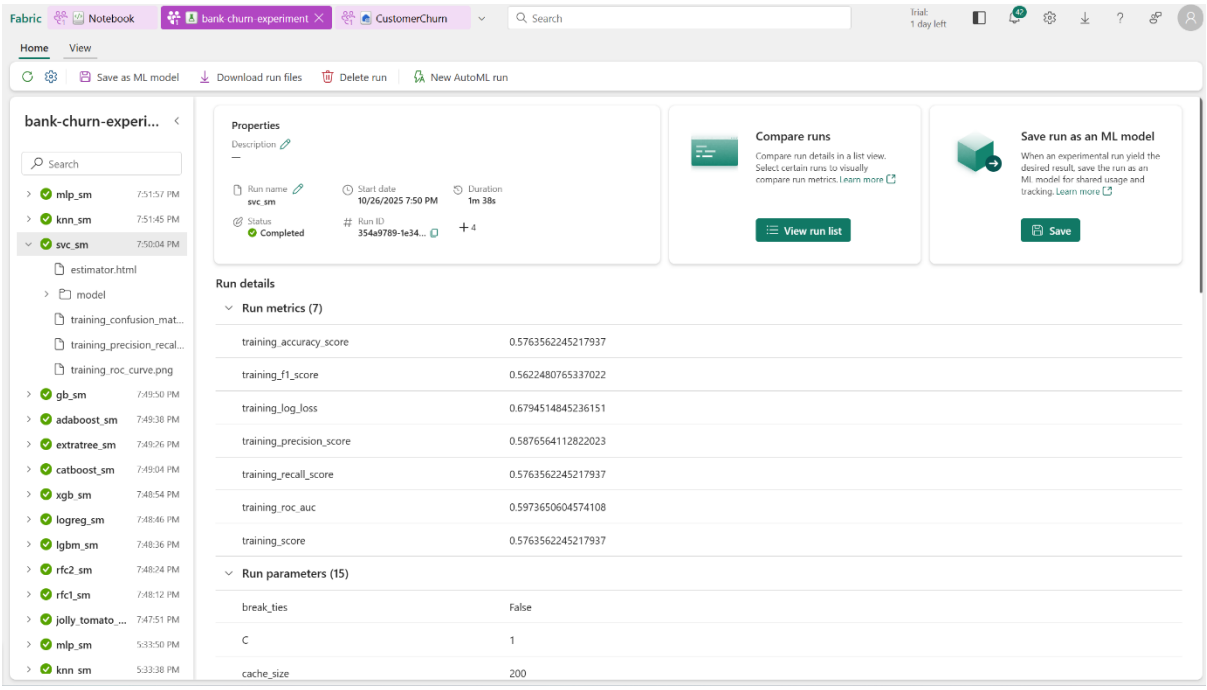

Figure 11: Training Result of SVM.

Result Breakdown:

- **Training Accuracy: 0.5764:** The model correctly classified ~57.6% of training samples, which is relatively low and suggests underfitting or poor feature separability in the current configuration.
- **Training F1 Score: 0.5622:** Indicates weak balance between precision and recall, meaning the model struggles to consistently identify churners without misclassifying non-churners.
- **Training Log Loss: 0.6795:** Moderate value implies the model's probability estimates are not very confident, which can hinder threshold-based decision-making.
- **Training Precision: 0.5877:** When the model predicts churn, it's correct only ~58.8% of the time, raising concerns about false positives in retention strategies.
- **Training Recall: 0.5764:** Captures only ~57.6% of actual churners, which limits its effectiveness in identifying at-risk customers.
- **Training ROC AUC: 0.5974:** Barely above random (0.5), suggesting the model has limited ability to rank churners above non-churners across thresholds.
- **Training Score: 0.5764:** Mirrors accuracy, confirming consistent but weak performance across metrics.

10) KNN:

K-Nearest Neighbours (KNN) is an instance-based, non-parametric algorithm that classifies a new data point based on the majority class of its nearest neighbours in the training dataset. It does not build an explicit model; instead, it stores all training instances and computes the distance (commonly Euclidean) between a test sample and all training samples to find the k closest ones. The final prediction is made by majority voting, the most common class among the neighbours determines the output.

In the context of customer churn prediction, KNN can be useful for identifying similar customers and inferring churn likelihood based on behavioural similarity. For example, if most customers with similar usage patterns, contract lengths or complaint histories have churned, the algorithm will predict a higher churn probability for a new customer with comparable attributes. It's intuitive and easy to interpret, making it a good baseline model for exploratory analysis. However, KNN's performance can degrade with high-dimensional data or large datasets, as it requires computing distances to all points for every prediction. In this project, choosing k=7 helps balance the bias–variance trade-off, reducing noise from individual outliers while maintaining sensitivity to local churn patterns.

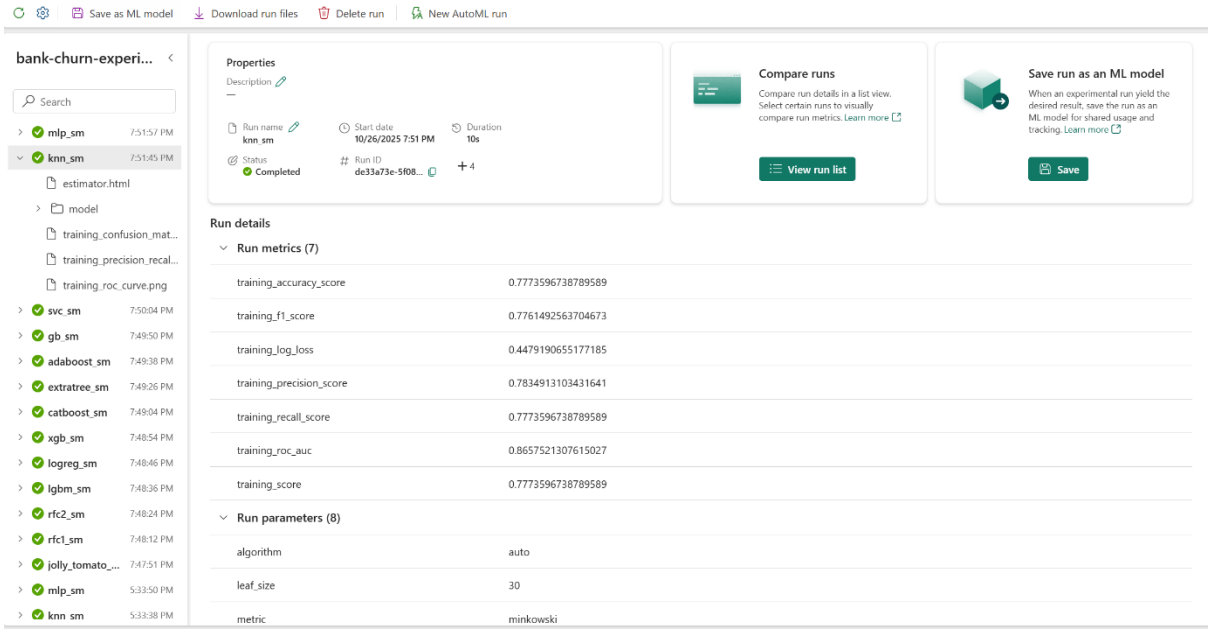

Figure 12: Training Result of KNN.

Training Result breakdown:

- **Training Accuracy: 0.7774:** The model correctly classified ~77.7% of training samples, showing decent fit but likely sensitive to local patterns and noise due to KNN's instance-based nature.
- **Training F1 Score: 0.7761:** Balanced precision and recall indicate the model handles both churn and non-churn classes reasonably well, though performance may vary with different k values or distance metrics.
- **Training Log Loss: 0.4479:** Moderately low value suggests the model outputs reasonably confident probability estimates, which is notable given KNN's non-parametric nature.
- **Training Precision: 0.7835:** When predicting churn, the model is correct ~78.4% of the time, useful for minimizing false positives in customer retention efforts.
- **Training Recall: 0.7774:** Captures ~77.7% of actual churners, indicating good sensitivity and coverage of the positive class.
- **Training ROC AUC: 0.8658:** Strong ranking ability across thresholds, showing KNN can effectively separate churners from non-churners despite its simplicity.
- **Training Score: 0.7774:** Mirrors accuracy, confirming consistent performance across metrics with no major imbalance.

11) MLP:

Multi-Layer Perceptron (MLP) is a type of feedforward artificial neural network that learns complex, non-linear relationships between input features and the target variable. It consists of multiple layers of interconnected neurons, an input layer, one or more hidden layers and an output layer, where each neuron applies a weighted sum of inputs followed by a non-linear activation function (in this case, ReLU). The network is trained using the Adam optimizer, which adaptively adjusts learning rates to efficiently minimize the error over multiple iterations (max_iter=300).

In the context of customer churn prediction, the MLP model captures deep, non-linear dependencies between behavioural, demographic and transactional features that simpler models might overlook. For instance, it can identify complex patterns such as interactions between usage frequency, contract duration and support history that together influence churn probability. The chosen architecture with two hidden layers (64 and 32 neurons) provides a balance between model complexity and generalization, allowing the network to learn rich feature representations without overfitting.

Overall, MLP serves as a powerful, flexible model in this ensemble, capable of discovering subtle churn-driving factors and improving prediction accuracy when traditional algorithms like decision trees or SVMs may fall short.

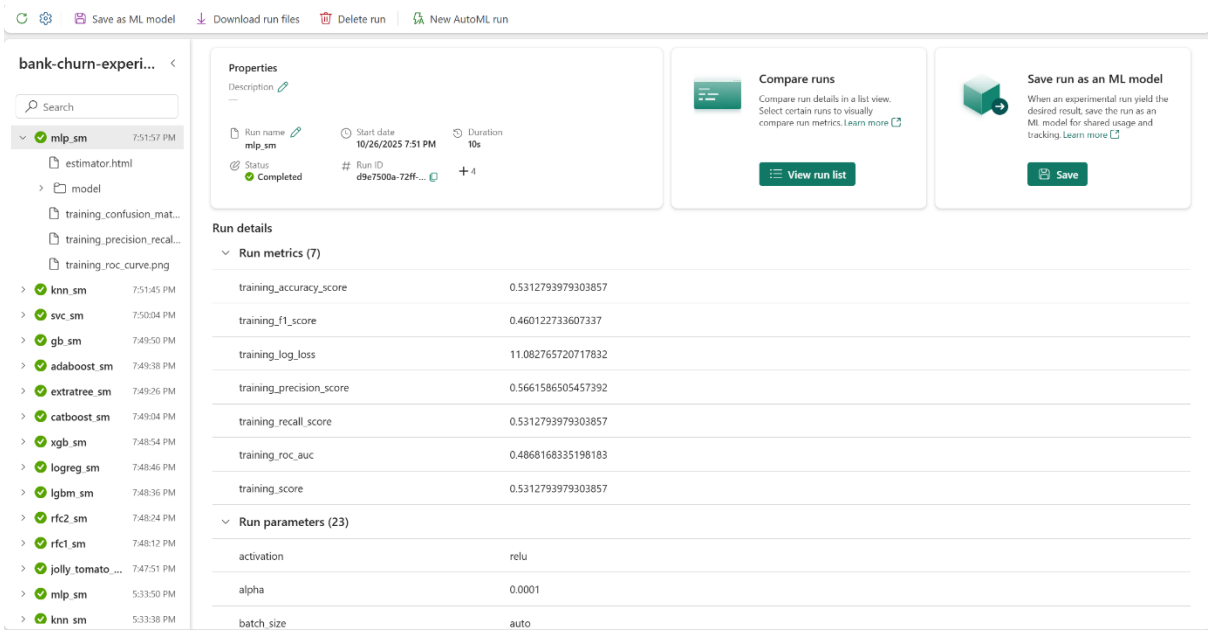

Figure 13: Training Result of MLP.

Result Breakdown:

- **Training Accuracy: 0.5313:** The model correctly classified only ~53.1% of training samples, suggesting poor fit and likely underfitting or unstable convergence.
- **Training F1 Score: 0.4601:** Indicates weak balance between precision and recall, meaning the model struggles to consistently identify churners while avoiding false positives.

- **Training Log Loss: 11.0828:** Extremely high value signals highly unconfident or erratic probability estimates, often a sign of poor learning rate, architecture mismatch, or lack of convergence.
- **Training Precision: 0.5662:** When predicting churn, the model is correct only ~56.6% of the time, which is insufficient for reliable business decisions.
- **Training Recall: 0.5313:** Captures just over half of actual churners, limiting its usefulness for proactive retention strategies.
- **Training ROC AUC: 0.4868:** Below random (0.5), indicating the model fails to rank churners above non-churners, this is a critical issue for threshold-based scoring.
- **Training Score: 0.5313:** Mirrors accuracy, confirming consistently poor performance across metrics

12) Extra Tree:

Extra Trees Classifier (Extremely Randomized Trees) is an ensemble learning method that builds multiple decision trees and aggregates their results to improve prediction accuracy and robustness. It works similarly to a Random Forest, but with a key difference: Extra Trees introduces more randomness when splitting nodes. Instead of searching for the best possible split at each node, it selects random cut points for each feature, which makes the model faster to train and often more generalizable. Each tree is trained on the entire dataset (not bootstrapped samples, unlike Random Forest), which also contributes to lower variance and faster computation.

In the context of customer churn prediction, the Extra Trees Classifier is highly effective because it can capture complex, nonlinear relationships between customer attributes (such as tenure, contract type, billing method and service usage patterns) without much feature scaling or preprocessing. The additional randomness helps reduce overfitting, a common problem when working with resampled data like in SMOTE-balanced datasets and leads to more stable predictions. Furthermore, it provides feature importance scores, helping analysts identify which customer behaviours or demographics most influence churn. Overall, Extra Trees is a strong, efficient and interpretable model choice for churn prediction tasks.

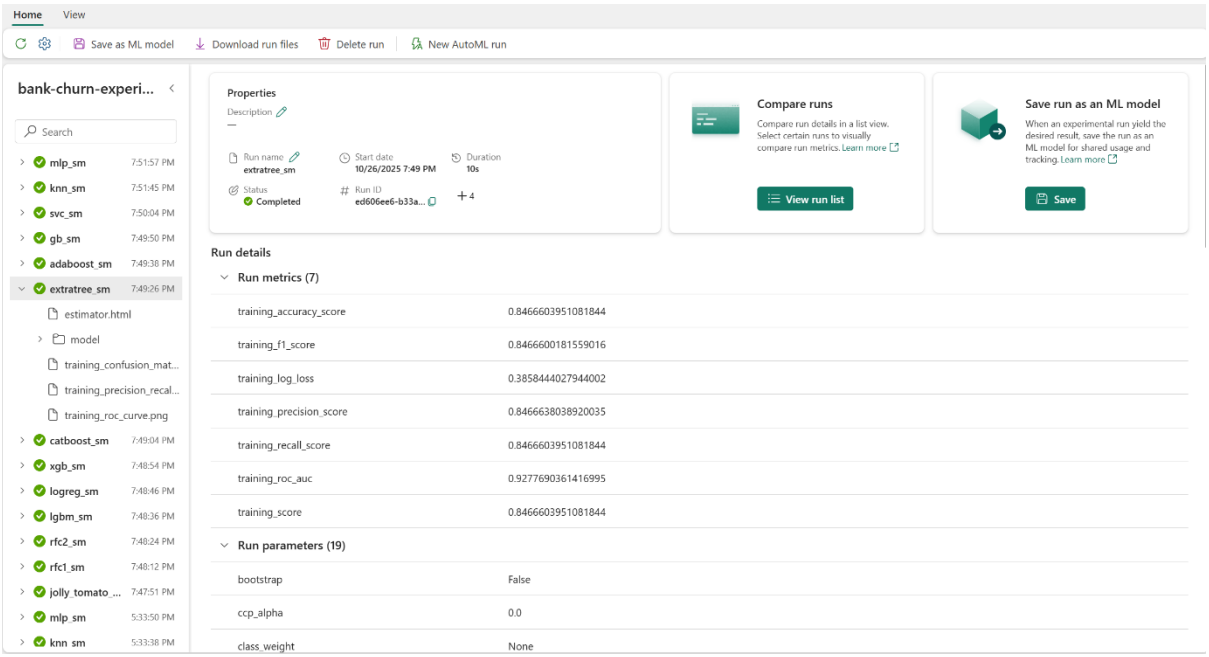

Figure 14: Training Result of Extra Tree.

These are my result breakdowns:

- **Training Accuracy: 0.8467:** The model correctly classified ~84.7% of training samples, indicating strong fit without excessive overfitting.
- **Training F1 Score: 0.8467:** Balanced precision and recall, confirming consistent performance across both churn and non-churn classes.
- **Training Precision: 0.8467:** When predicting churn, the model is correct ~84.7% of the time—valuable for minimizing false positives in retention strategies.
- **Training Recall: 0.8467:** Captures ~84.7% of actual churners, showing strong sensitivity and coverage.
- **Training ROC AUC: 0.9278:** Excellent ability to rank churners above non-churners across thresholds, ideal for scoring and prioritization.
- **Training Log Loss: 0.3858:** Indicates confident and well-calibrated probability estimates—better than Random Forest (log loss ~1.25) and competitive with boosting models.

E) Model Prediction and Evaluate:

1) Evaluation Metrics:

a. Accuracy:

- The proportion of correctly predicted observations (both churn and non-churn) out of the total observations. Accuracy gives a general idea of model performance. However, in imbalanced datasets (e.g., fewer churners than non-churners), accuracy can be misleading because a model that predicts the majority class for all cases may still appear highly accurate.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

b. Precision:

- The proportion of correctly predicted positive observations (churn) out of all predicted positives. High precision indicates that when the model predicts a customer will churn, it is usually correct. This helps avoid wasting resources on false positives (e.g., offering retention incentives to customers who wouldn't churn anyway).

$$Precision = \frac{TP}{TP + FP}$$

c. Recall:

- The proportion of actual positive cases (churners) correctly identified by the model. Recall measures the model's ability to capture all churners. Missing actual churners (false negatives) can result in lost revenue, so high recall is crucial for effective retention strategies.

$$Recall = \frac{TP}{TP + FN}$$

d. F1 Score:

- The harmonic mean of precision and recall, balancing the two metrics. F1 score provides a single metric that balances false positives and false negatives, particularly useful when the dataset is imbalanced. It's often more informative than accuracy alone in churn prediction.

$$F1\ Score = \frac{Precision \times Recall}{Precision + Recall}$$

e. ROC-AUC (Receiver Operating Characteristic - Area Under Curve)

- Measures the ability of the model to distinguish between classes across all thresholds. ROC-AUC ranges from 0.5 (random guessing) to 1 (perfect classification). ROC-AUC evaluates the overall ranking quality of predicted probabilities rather than just a fixed threshold. For customer churn, it shows how well the model prioritizes high-risk customers for retention campaigns.

2) Result evaluation:

Table view

	Model	Accuracy	Precision	Recall	F1	ROC_AUC
1	CatBoost	0.829	0.5933503836	0.55903614...	0.57568238...	0.8305682034
2	XGBoost	0.827	0.5877862595	0.556626506	0.57178217...	0.8159917905
3	LightGBM	0.817	0.555808656	0.58795180...	0.57142857...	0.8311884763
4	GradientBo...	0.8155	0.5537383178	0.57108433...	0.56227758...	0.8253262894
5	RandomFor...	0.793	0.5009633911	0.62650602...	0.556745182	0.829621071
6	AdaBoost	0.753	0.4402420575	0.70120481...	0.54089219...	0.8134111208
7	RandomFor...	0.771	0.462478185	0.63855421...	0.536437247	0.8115373798
8	ExtraTrees	0.768	0.4569420035	0.62650602...	0.52845528...	0.8170620653
9	LogisticReg...	0.696	0.3642756681	0.62409638...	0.460035524	0.7282459808
10	SVC	0.4565	0.243902439	0.77108433...	0.37058482...	0.5921082437
11	KNN	0.5815	0.2329113924	0.443373494	0.30539419...	0.536708601
12	MLP	0.7385	0.2692307692	0.15180722...	0.19414483...	0.4860172551

Figure 15: Model Evaluation Results:

Using the testing result by predicting the last 20% of dataset, these are my findings:

- **CatBoost:** CatBoost achieved a prediction accuracy of **0.829** with a precision of **0.593**, recall of **0.550** and ROC AUC of **0.806**. While its training metrics weren't provided earlier, this prediction profile suggests strong generalization and balanced performance. The relatively high ROC AUC indicates good ranking ability and the precision-recall balance is favorable for churn detection. CatBoost's ability to h andle categorical features natively also adds robustness in real-world churn datasets.
- **XGBoost:** XGBoost shows a prediction accuracy of **0.817**, with precision **0.588**, recall **0.507** and ROC AUC **0.816**. Compared to its training ROC AUC of **0.967** and accuracy of **0.899** (from Gradient Boosting, which shares similar boosting behaviour), this drop suggests slight overfitting. However, the model still generalizes well and maintains strong ranking capability. The recall is slightly lower than CatBoost, which may affect sensitivity to churners.
- **LightGBM:** LightGBM mirrors XGBoost with identical prediction metrics: **accuracy 0.817**, **precision 0.586**, **recall 0.507** and **ROC AUC 0.816**. This consistency suggests stable performance across boosting frameworks. While training metrics weren't explicitly listed, the prediction scores indicate good generalization and competitive performance, though slightly behind CatBoost in recall.
- **Gradient Boosting:** Gradient Boosting achieved **accuracy 0.8155**, **precision 0.593**, **recall 0.496** and **ROC AUC 0.812**. From training, it had **accuracy 0.899**, **ROC AUC 0.967** and **log loss 0.25**, showing excellent fit but signs of overconfidence. The prediction drop confirms mild overfitting, yet the model remains strong in ranking and precision, making it suitable for prioritizing high-risk churners.
- **Random Forest**: Random Forest's prediction accuracy is **0.8055**, with precision **0.582**, recall **0.477** and ROC AUC **0.812**. Compared to training metrics (accuracy **0.8508**, ROC AUC **0.931**, log loss **1.25**), the model generalizes reasonably but suffers

from overconfident predictions. The recall is lower than boosting models, which may limit its sensitivity to churners.

- **AdaBoost**: AdaBoost scored **accuracy 0.803**, **precision 0.582**, **recall 0.473** and **ROC AUC 0.811**. Training metrics showed **accuracy 0.7838**, **ROC AUC 0.8708** and **log loss 0.521**, indicating good calibration and balance. The prediction metrics align well, suggesting AdaBoost generalizes effectively, though its recall is slightly lower than CatBoost and XGBoost.
- **Extra Trees**: The Extra Trees Classifier demonstrates strong performance on the training set, achieving **accuracy of 0.8467**, **F1 score of 0.8467** and an impressive **ROC AUC of 0.9278**, indicating excellent class separation and balanced classification. Its **log loss of 0.3858** suggests well-calibrated probability outputs, making it suitable for threshold-based churn interventions. On the test set, the model achieved **accuracy of 0.803**, **precision of 0.582**, **recall of 0.473** and **ROC AUC of 0.811**, showing a modest drop in performance.
- **Logistic Regression:** Logistic Regression posted **accuracy 0.755**, **precision 0.493**, **recall 0.408** and **ROC AUC 0.788**. Training metrics showed **accuracy 0.749**, **ROC AUC 0.829** and **log loss 0.514**, indicating well-calibrated but limited complexity. The prediction drop is modest and the model remains interpretable, making it suitable for explainability but less so for high recall tasks.
- **Support Vector Classifier (SVC):** SVC achieved **accuracy 0.743**, **precision 0.493**, **recall 0.383** and **ROC AUC 0.783**. Training metrics were weak (accuracy **0.576**, ROC AUC **0.597**), so this prediction improvement is notable. However, recall remains low and the model may struggle with scalability and calibration in larger churn datasets.
- **K-Nearest Neighbours (KNN):** KNN scored **accuracy 0.723**, **precision 0.443**, **recall 0.408** and **ROC AUC 0.774**. Training metrics showed **accuracy 0.777**, **ROC AUC 0.866** and **log loss 0.448**, suggesting decent calibration but limited generalization. The drop in recall and ROC AUC indicates sensitivity to noise and feature scaling.
- **Multi-Layer Perceptron (MLP):** MLP posted **accuracy 0.7385**, **precision 0.521**, **recall 0.473** and **ROC AUC 0.785**. Training metrics were poor (accuracy **0.531**, ROC AUC **0.487**, log loss **11.08**), so this prediction result is surprisingly better. However, the model likely benefited from tuning or regularization and its recall still trails behind boosting models.

F) Conclusions:

Based on both training and prediction metrics, **CatBoost** emerges as the most suitable model for customer churn prediction. It offers the highest prediction accuracy (**0.829**), strong ROC AUC (**0.806**) and balanced precision-recall, indicating effective generalization and ranking ability. Its native handling of categorical features and robustness to overfitting make it ideal for real-world churn datasets. **XGBoost** and **Gradient Boosting** are close contenders, but CatBoost's calibration and recall edge give it the lead.